

A Remote FPGA Infrastructure for Training and Evaluating Agentic Reasoning Tasks in the Physical World

Design Iteration One — Manhattan Reasoning

2026-06-02

Table of contents

1	Purpose and Scope	3
1.1	What the document covers:	3
1.2	What the document does not cover:	3
2	Background & Motivation	3
2.1	Problem Statement	3
2.2	Goals	4
2.3	Non-Goals	4
3	System Overview	4
3.1	Physical Layer	5
3.2	Hardware Firmware Layer	5
3.3	Host Layer	5
3.4	User Interface Layer	5
4	Physical Layer	5
4.1	FPGA Node Specification	5
4.2	Power Specification	6
4.3	Ethernet Connectivity	7
4.4	Local Network Topology	7
4.5	USB / JTAG Topology	8
4.6	UART Topology	8
5	Hardware Firmware Layer	8
5.1	System Components	8
5.2	Resource Utilization	9
5.2.1	Logic (Yosys post-synthesis, ECP5 primitives)	9
5.2.2	Block Resources (nextpnr post-PnR, confirmed placed)	10
5.2.3	Timing (nextpnr static analysis, no timing constraints imposed)	10

5.2.4	Summary	11
5.3	Wishbone Interface	11
5.4	Firmware Purpose	13
5.5	Toolchain	13
5.5.1	Synthesis, Place-and-Route, Bitstream	13
5.5.2	Flashing to Hardware	13
6	Host Layer	14
6.1	API Server	14
6.2	Job Queue	15
6.2.1	Job Types	15
6.2.2	Workers	15
6.2.3	build_and_program	16
6.2.4	run	16
6.2.5	reset	17
6.2.6	Worker Resources	17
6.3	Build Server	17
6.3.1	Sandboxing	18
6.4	JTAG Programmer	18
6.5	Runtime Bridge	19
7	Networking	19
7.1	Local LAN Topology	19
7.1.1	ARP	20
7.1.2	FPGA Interconnections	20
7.1.3	Firewall	20
7.2	Remote Access	20
7.2.1	DigitalOcean Droplet	20
7.2.2	TailScale	21
7.3	Runtime Datapath	21
7.3.1	To FPGA	21
7.3.2	From FPGA	21
7.4	Protocols and Ports	22
8	User Interface Layer	22
8.1	HDL Interface Contracts	22
8.2	User Submission Contracts	23
8.3	User Return Contracts	23
8.4	Runtime Contracts	23
9	Infrastructure Does Not Guarantee	23
	References	24

1 Purpose and Scope

This document details the technical design of an open-source FPGA cloud platform targeted at training and evaluating agents in reasoning about physical environments, specifically electrical engineering tasks. This marks the first design iteration.

1.1 What the document covers:

- Physical Hardware: 10-node ECP5-5G cluster and physical connections.
- Hardware Firmware: LiteX SoC with soft-core ethernet and CPU on each FPGA.
- Host Software: The API server that permits FPGA programming and runtime dispatch.
- Networking: Local LAN topology and remote user access.
- User Interface: How Amaranth HDL designs are submitted and executed.

1.2 What the document does not cover:

- Agent Competition: This document solely focuses on infrastructure. Users are assumed to be any entity that can make requests. Overlaying competition infrastructure is not in scope.
- Automated Research: General-purpose agentic evaluation is not covered in this document.
- Billing, authentication, multi-tenancy policy: SaaS-related constructs are deferred. This document pertains only to basic infrastructure, serving one user.

2 Background & Motivation

2.1 Problem Statement

With artificial intelligence becoming increasingly better with the advent of LLM-based systems, automated scientific discovery has become a real and tangible goal. For example, Liu et al. illustrates AutoResearchClaw, an autonomous research system that is capable of discovering literature, evaluating its own ideas, performing iterative experimentation, writing reports, and verifying its results [4]. However, there remains a gap in LLM-based automated research in physical environments such as Electronic Design Automation (EDA) tasks pertaining to FPGA-based development. Traditional SWE-based agents do not perform as well in hardware environments [9].

Cloud FPGA infrastructure provides a safe and accessible environment for agents to learn how to reason about physical engineering decisions. Amazon EC2 F2 Instances provide industry-grade cloud FPGA resources, however their use of robust proprietary software often leads to long delays in FPGA usage, posing a significant barrier to agentic systems that often require quick feedback for iterative refinement [1].

We propose a cloud infrastructure containing FPGAs built on solely open-source and lightweight toolchains, specifically targeting agentic evaluation in physical engineering tasks rather than industry-grade FPGA prototyping. Our initial design will focus on agents' ability to design automated reasoning tools, specifically boolean satisfiability solvers, a complex task motivated by the success of stand-alone hardware solvers such as SAT-Accel proposed by Lo et al. and SAT-Hard proposed by Ustaoglu et al. [5, 8].

2.2 Goals

- Fully Open Source Infrastructure:
 - FPGA toolchain: Yosys, Nextpnr
 - HDL: Amaranth
 - SoC: LiteX
- Straightforward User Experience: User can easily program and run FPGA.
- Scalable to 10 independent FPGA nodes.
- Accessible remotely over HTTPS.

2.3 Non-Goals

- Distributed System: The FPGA's are independent and cannot communicate with one another.

3 System Overview

The proposed infrastructure contains 10 [Lattice ECP5 Evaluation Boards](#), each with an ECP5-5G FPGA (LFE5UM5G-85F-8BG381). Each board serves as a node in our local network of FPGAs. Each FPGA individually connects to a shared host machine via USB/JTAG for programming and ethernet for runtime interfacing. The host machine handles API requests, allowing users to request resource access (programming and running selected FPGAs). A [Tailscale](#) tunnel provides a connection from the host system to a [DigitalOcean](#) Droplet that contains a Linux virtual machine. This acts as a reverse proxy that accepts HTTPS requests from a user, enabling resource utilization.

The system is divided into four layers. This section details a one-paragraph mental model for each. Later sections cover detailed descriptions.

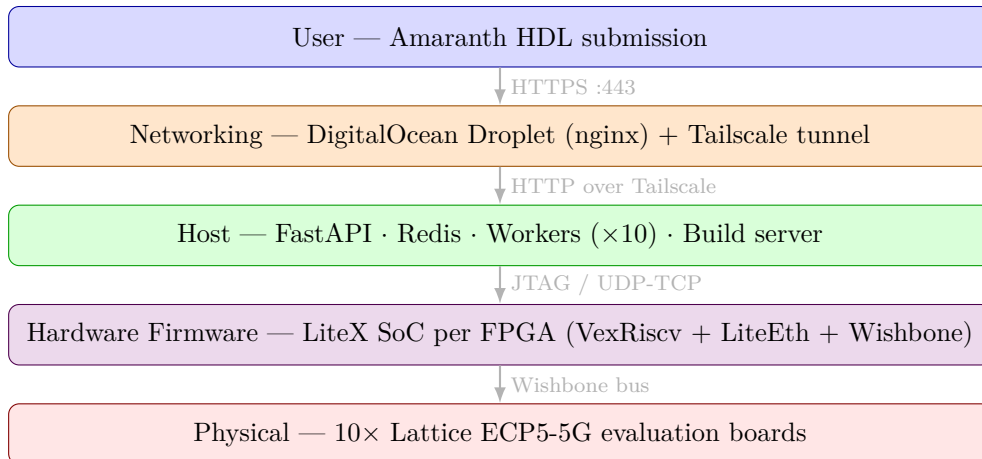


Figure 1: Four-layer system architecture

3.1 Physical Layer

Ten Lattice ECP5-5G evaluation boards each serve as an independent FPGA node. Each board draws power from a shared PDU and connects to the host machine over two interfaces: a mini USB port (via an FTDI chip for JTAG programming) and an ethernet port (via a LAN8720 PHY board for runtime data transfer). All boards share an unmanaged ethernet switch and a powered USB hub that fan out to the host over single cables.

3.2 Hardware Firmware Layer

Each FPGA runs a LiteX SoC consisting of a VexRiscv soft-core CPU, a LiteEth MAC and DMA for ethernet, a Wishbone bus for memory-mapped I/O, and on-chip ROM and SRAM. Bare-metal firmware on the CPU handles ethernet initialization and routes incoming packets to user-defined memory regions via the Wishbone bus. The full toolchain — Yosys, nextpnr-ecp5, and ecppack — synthesizes, places and routes, and packs bitstreams from LiteX-generated Verilog. The SoC currently consumes roughly 6% of logic and 11% of block RAM, leaving the vast majority of the chip free for user designs.

3.3 Host Layer

A FastAPI server running on the host machine exposes a REST API over HTTPS that accepts HDL submissions and runtime requests. Jobs are queued per FPGA in Redis and executed by per-FPGA worker processes. The build pipeline converts a user’s Amaranth design to Verilog, merges it with the base LiteX SoC, runs the full synthesis toolchain, and flashes the resulting bitstream via openFPGALoader over JTAG. A runtime bridge opens a UDP or TCP socket to the programmed FPGA to send input payloads and collect results. Remote access is handled through a DigitalOcean Droplet acting as an nginx reverse proxy, connected to the host via a Tailscale tunnel.

3.4 User Interface Layer

Users submit a single Amaranth .py file whose top-level module conforms to a defined Wishbone B4 slave interface contract. The API returns a session ID on successful programming; subsequent run requests reference that session to send data to and receive results from the live FPGA. Contracts specify the required module structure, sandbox restrictions (no network or filesystem access during build), and the full set of success and failure response shapes. A final section enumerates what the infrastructure explicitly does not guarantee, including design persistence, bitstream retention, FPGA availability, and partial results on timeout.

4 Physical Layer

4.1 FPGA Node Specification

Parameter	Value
Device	Lattice LFE5UM5G-85F-8BG381 (ECP5-5G)
Board	Lattice ECP5 Evaluation Board
LUTS	84,000
BRAM	3744 kbits
Power	External 12V Power Supply (3A DC barrel adapter)
Programming Interface	FT2232H FTDI converts USB to JTAG
Interface Port	mini USB

4.2 Power Specification

Each evaluation board comes with a 12V AC/DC power adapter. We have two potential designs:

Parameter	Value
Rack PDU (Power Distribution Unit)	10 12V AC/DC adapters are powered through a PDU
DIN-rail PSU + distribution block	PSU that converts wall voltage to 12V. Distribution block splits 12V input into 10 outputs with 2.1mm barrel connector. Requires DIN rail.

Derived from [3] the following power consumption per rail is derived.

Rail	V	Max I (A)	Output Watts
3.3 V I/O	3.3	1.35	4.46
2.5 V I/O	2.5	1.10	2.75
VCC_CORE	1.2	1.35	1.62
1.5 V logic	1.5	1.10	1.65
SERDES VCCHTX0	1.2	0.50	0.60
SERDES VCCHTX1	1.2	0.50	0.60
SERDES VCCA0	1.2	0.50	0.60
SERDES VCCA1	1.2	0.50	0.60
Total			12.88

Taking into consideration efficiency [2] with some estimation based on given plots, the following power consumption per FPGA is derived.

Rail	V	Max I (A)	Output Watts	Efficiency	Input Watts
3.3 V I/O	3.3	1.35	4.46	85%	5.25
2.5 V I/O	2.5	1.10	2.75	82%	3.35
VCC_CORE	1.2	1.35	1.62	72%	2.25
1.5 V logic	1.5	1.10	1.65	75%	2.20
SERDES VCCHTX0	1.2	0.50	0.60	72%	0.83

Rail	V	Max I (A)	Output Watts	Efficiency	Input Watts
SERDES VCCHTX1	1.2	0.50	0.60	72%	0.83
SERDES VCCA0	1.2	0.50	0.60	72%	0.83
SERDES VCCA1	1.2	0.50	0.60	72%	0.83
Total			12.88		16.37

We consider USB power requirements per FPGA. We require a powered 10-port USB hub.

4.3 Ethernet Connectivity

The Lattice LFE5UM5G-85F-8BG381 does not contain a hard-core MAC. A LiteX MAC soft-core will be explained in the hardware firmware section.

10 [LAN8720 Ethernet Boards](#) will individually bridge each FPGA evaluation board to ethernet. The LAN8720 uses a single-chip ethernet physical layer transceiver (PHY) chip to convert between analog and digital signals. It talks to the GPIO pins on the FPGA via Reduced Media Independent Interface (RMII). It supports 100 Mbps operation [6].

4.4 Local Network Topology

FPGAs will connect over ethernet to an unmanaged ethernet switch that connects to the host machine. An [unmanaged switch](#) allows for plug and play, sufficient for our first design, only involving 10 FPGAs. They are cheaper, however, lack the same security guarantees as managed switches. The ethernet switch will need to support 11 ethernet cables. One ethernet input from the host. 10 ethernet outputs to the FPGAs.

Each device on the local network will contain an unique IP address. For our small initial setup, we will use hardcoded IP addresses defined in SoC firmware at start-up. IP address range will be defined as 192.168.1.101 to 192.168.1.110, a standard home-network naming convention. The host machine will require a FPGA-facing interface 192.168.1.10 that will need to be configured via [Netplan](#) (Assumes Ubuntu).

Each device on the local network will contain an unique MAC address. The first byte of the MAC address contains a multicast bit set to 0 and a locally administered bit set to 1, yielding the 02:xx:xx:xx:xx:xx convention. We arbitrarily set the addresses of each FPGA as 02:00:00:00:00:01 through 02:00:00:00:00:0A. The host should have a pre-defined MAC address.

An open question is whether to use UDP or TCP. The maximum payload for an UDP packet is 65KB. We need to consider if the FPGA can tolerate what amount of clauses and variables. For a general purpose, long-term solution, we want TCP. For starting design with minimal complexity, UDP is simpler to configure over LiteX.

4.5 USB / JTAG Topology

Each FPGA has a mini USB port and will individually be connected to a powered USB hub. This hub connects to the host machine over one USB cable.

On the FPGA end, an FTDI chip translates USB into JTAG protocol signals. This is used to program the FPGA.

On the host end, each FPGA is a separate USB device. [ftdi_eeeprom](#) programs each FPGA's serial number to a unique value. A custom Linux udev rule will enumerate each device i.e. `/dev/fpga00` through `/dev/fpga09`, mapping to each serial number.

4.6 UART Topology

A [CP2102 USB 2.0 to TTL module serial converter](#) will bridge each FPGA to USB that will connect to the same powered USB hub. Linux will interface in the same way as in USB/JTAG.

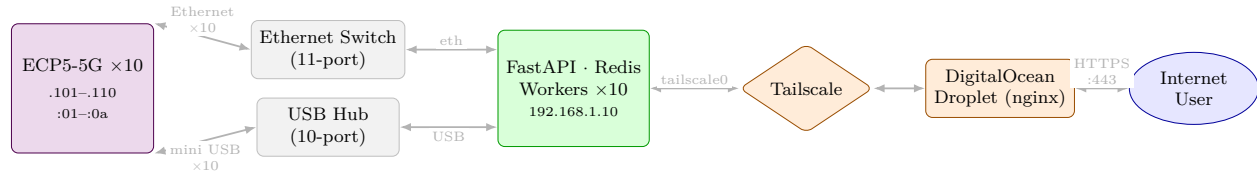


Figure 2: Physical cluster topology

5 Hardware Firmware Layer

5.1 System Components

Each FPGA will run a [LiteX](#) System on Chip (SoC).

The proposed LiteX SoC requires the following components:

#	Component	Description
1	VexRiscv CPU	The standard 32-bit VexRiscv soft core CPU. We opt to use the higher resource standard version over the smaller variants due to cache inclusion. This increases performance needed for meeting 100Mbps transfer speeds.
2	LiteEth MAC + DMA Ethernet Module	Offers configurable MAC interface in hardware supporting MII and RMII 100Mbps PHYs. Writes received data to SRAM.
3	Wishbone Bus	32-bit, memory-mapped bus.
4	LiteX UART Module	We use a provided UART module from LiteX to load bare metal firmware for the CPU to control the ethernet module.
5	32 KB ROM	Stores boot firmware.

#	Component	Description
6	16 KB SRAM	General-purpose on-chip RAM.
7	ECP5 PLL (12 MHz $\rightarrow$ 50 MHz)	On-chip phase-locked loop that scales the 12 MHz board oscillator up to the 50 MHz system clock.

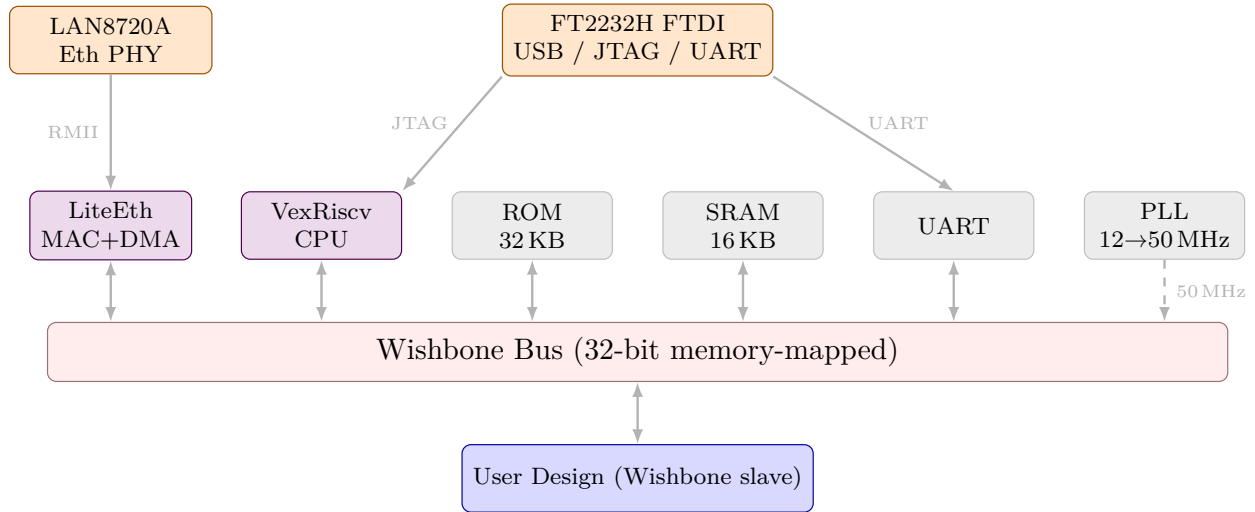


Figure 3: LiteX SoC block diagram — one instance per FPGA

⚠ Open Design Decision

ROM and SRAM sizes remain undecided. The values used here were chosen for default prototyping and are reflected in the resource utilization figures below.

The resource utilization and timing summary are as follows:

5.2 Resource Utilization

5.2.1 Logic (Yosys post-synthesis, ECP5 primitives)

Primitive	Used	Notes
LUT4	4,601	Combinational logic across all SoC modules
TRELLIS_FF	2,703	Flip-flops (pipeline registers, state machines, FIFOs)
CCU2C	371	Carry-chain arithmetic (adders, comparators)
PFUMX / L6MUX21	841	Wide multiplexers (Wishbone interconnect, datapath)

Primitive	Used	Notes
TRELLIS_DPR16X4	55	Distributed 16×4 RAM (small FIFOs, shift registers)

5.2.2 Block Resources (nextpnr post-PnR, confirmed placed)

Resource	Used	Total	% Used	Remaining	Notes
TREL- LIS_COMB (LUT4+mux)	5,831	83,640	6%	77,809	Physical logic cells placed
TRELLIS_FF	2,697	83,640	3%	80,943	Physical flip-flops placed
TREL- LIS_RAMW	55	10,455	0%	10,400	Distributed RAM words
DP16KD (EBR)	24	208	11%	184	ROM ×4, SRAM ×2, I-cache ×4, D-cache ×4, MAC buffers ×8, CSR SRAM ×2
MULT18X18D (DSP)	4	156	2%	152	VexRiscv hardware multiply/divide unit
EHXPLL (PLL)	1	4	25%	3	CRG: 12 MHz → 50 MHz sys + 50 MHz eth
IOLOGIC	7	224	3%	217	RMII DDR ref-clk output + RX/TX pads
DCCA	2	56	3%	54	Global clock distribution (sys + eth)
TRELLIS_IO	14	365	3%	351	Physical I/O pins used

5.2.3 Timing (nextpnr static analysis, no timing constraints imposed)

Clock domain	Target	Max achievable	Headroom
cd_sys (system)	50 MHz	126.77 MHz	2.5×
cd_eth (RMII)	50 MHz	151.38 MHz	3.0×

5.2.4 Summary

Metric	Value
Logic cells used	6% of chip
Block RAMs used	11% of chip (24 / 208 EBRs)
DSPs used	2% of chip (4 / 156)
Logic cells remaining	~77,800 / 83,640 (~94% free)
EBRs remaining	184 / 208 (88% free)
DSPs remaining	152 / 156 (97% free)

5.3 Wishbone Interface

The following diagram illustrates common signals for a Wishbone interface:

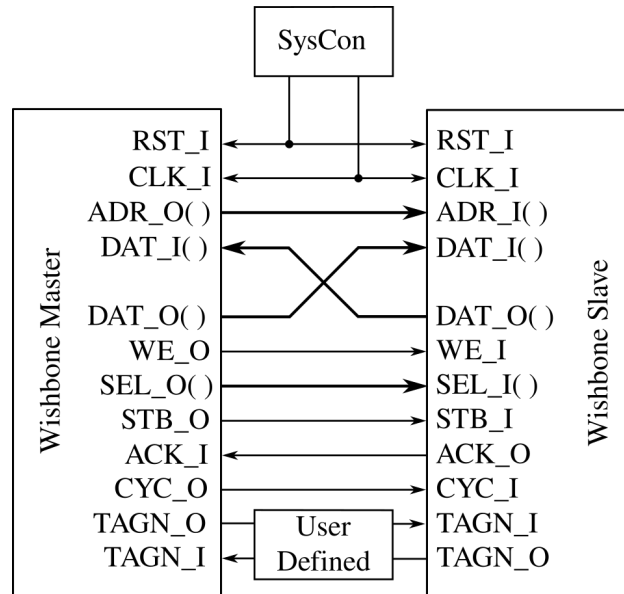


Figure 4: Wishbone Interface Diagram

Signal naming follows the B4 convention [7]: suffix **_O** = driven by that component, **_I** = received by that component. The same physical wire appears as e.g. **ACK_O** on the slave and **ACK_I** on the master.

Shared (driven by SysCon)

Signal	Width	Description
CLK_I	1	System clock. All transfers are synchronous to the rising edge.
RST_I	1	Synchronous reset, active high.

Master → Slave

Master	Slave	Width	Description
ADR_O()	ADR_I()	32	Binary address passed to the slave.
DAT_O()	DAT_I()	32	Data from master to slave (write data).
WE_O	WE_I	1	Write enable. Asserted during WRITE cycles, negated during READ cycles.
SEL_O()	SEL_I()	4	Byte select. Each bit enables one byte lane of DAT (one bit per byte for 32-bit/byte-granularity).
STB_O	STB_I	1	Strobe. Indicates a valid data transfer cycle. Slave must respond with ACK.
CYC_O	CYC_I	1	Cycle. Asserted for the entire duration of a bus transaction.
TAGN_O()	TAGN_I()	var	User-defined tag. Carries metadata associated with the cycle.

Slave → Master

Slave	Master	Width	Description
DAT_O()	DAT_I()	32	Data from slave to master (read data).
ACK_O	ACK_I	1	Acknowledge. Asserted to signal normal termination of a bus cycle.
TAGN_O()	TAGN_I()	var	User-defined tag response.

Users should have access to any memory address outside reserved regions for the SoC.

Open Design Decision

Whether to further constrain the user-accessible address range is undecided.

5.4 Firmware Purpose

The firmware should handle ethernet hardware initialization and packet routing to user-defined memory regions via the Wishbone bus.

A sample payload in a TCP/UDP packet concept:

Byte Offset	Size (bytes)	Field	Meaning
0	1	opcode	0x01 = write, 0x02 = read
1	3	length	Number of 32-bit words following
4	4	address	Wishbone offset
8	4	data[0]	First 32-bit word to write
12	4	data[1]	Second word (if length > 1)
8 + 4×n	4	data[n]	nth 32-bit word

5.5 Toolchain

5.5.1 Synthesis, Place-and-Route, Bitstream

Tool	Role
Yosys	Synthesis — converts LiteX-generated Verilog to ECP5 gate-level netlist
nextpnr-ecp5	Place and route — maps netlist to physical ECP5 resources, produces <code>.config</code>
ecppack	Bitstream packer — converts nextpnr <code>.config</code> to flashable <code>.bit</code> file
Project Trellis	ECP5 bitstream database used by nextpnr and ecppack
LiteX + Migen	SoC generator — produces the Verilog that Yosys consumes
LiteEth	Ethernet RTL cores (RMII PHY, MAC, DMA)

5.5.2 Flashing to Hardware

Tool	Role
OpenFPGALoader	Drives the onboard FTDI FT2232H over USB to load the bitstream via JTAG

6 Host Layer

The host machine orchestrates the entire cluster. It exposes an HTTPS API to users and handles requests to program and send data back and forth from FPGAs.

[FastAPI](#) will be used for our REST API which will be explained in the following sections.

[Redis](#) or Remote Dictionary Server will be used to manage a job queue, FPGA state, returning results/logging, and session state. These concepts will be explained in the following sections.

6.1 API Server

We will use a REST HTTP service. Specifically, we will use FastAPI. An API key likely makes sense but is reserved as a later decision choice. The initial prototype will be unauthenticated.

Method	Endpoint	Description	Session Auth	Sync / Async
GET	/fpga	List all 10 FPGAs and their current state	API key	Sync
GET	/fpga/{id}	Get state, current job, and session info for one FPGA	API key	Sync
POST	/fpga/{id}/submit	Upload HDL source, trigger build and program pipeline	API key	Async — returns job ID (202)
GET	/fpga/{id}/jobs/{job_id}	Poll build/program job status and current phase	API key, owner only	Sync
GET	/fpga/{id}/jobs/{job_id}/logs	Retrieve build logs including Yosys and nextpnr output	API key, owner only	Sync
DELETE	/fpga/{id}/jobs/{job_id}	Cancel a queued job before it starts building	API key, owner only	Sync
POST	/fpga/{id}/run	Send input data to running design, receive result	API key + session ID, owner only	Async — returns job ID (202)
GET	/fpga/{id}/session	Check active session, owner, and remaining TTL	API key + session ID, owner only	Sync
POST	/fpga/{id}/session/release	Explicitly release FPGA reservation when done	API key + session ID, owner only	Sync
POST	/fpga/{id}/reset	Reflash base SoC, clear user design, return to idle	API key + session ID (owner) or admin key	Async — returns job ID (202)
GET	/health	Check host, Redis, and all FPGA network reachability	None	Sync

API Conventions

Synchronous endpoints block until complete; asynchronous endpoints return a job ID (HTTP 202) immediately. When a user programs an FPGA a session is created and tied to that user — endpoints marked *session ID* require an active session, and *owner only* endpoints are restricted to the submitting user. TTL (Time-to-Live) is the remaining lifetime of a session before it expires and the FPGA returns to idle.

⚠ Deferred Design Decision

POST /run may be replaced by a persistent bidirectional connection in a future iteration.

6.2 Job Queue

For the simplicity of our initial design, each FPGA will have an independent job queue. Jobs on different FPGAs can run independently and concurrently.

⚠ Open Design Decision

The depth of each per-FPGA job queue is undecided.

6.2.1 Job Types

There are three core job types:

Job type	Description
build_and_program	Full pipeline: compile HDL source through Yosys + nextpnr, produce a bitstream, flash it to the FPGA over JTAG, wait for the board to come online, create a session for the submitting user
run	Send a data payload to a reserved FPGA over UDP/TCP, wait for the result, return it to the caller. Requires an active session owned by the submitting user
reset	Flash the base LiteX SoC bitstream to the FPGA, clearing any user design. Invalidates the active session and returns the FPGA to idle

⚠ Open Design Decision

Timeout durations per job type and error recovery behavior are undecided.

6.2.2 Workers

Each job queue will be managed by a worker: a long-running OS process on the host machine. Workers complete jobs as they arrive. Workers start automatically and will need to gracefully restart on failure.

Workers progress FPGA state. The diagram below shows all states and transitions; the tables that follow detail each job type individually.

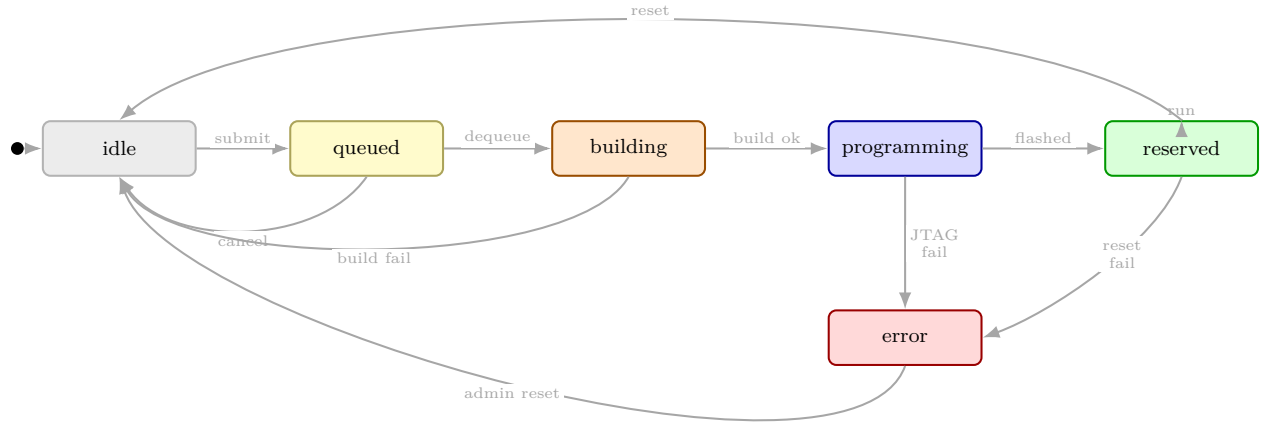


Figure 5: FPGA state machine

6.2.3 build_and_program

FPGA state	Triggered by	Next state on success	Next state on failure
idle	Job accepted into queue	queued	—
queued	Worker picks up job	building	idle (job cancelled before start)
building	Worker runs Yosys + nextpnr	programming	idle (build error is user fault, board untouched)
programming	Worker runs openFPGALoader over JTAG	reserved	error (JTAG failed, hardware suspect)
reserved	Board online, session created for owner	—	—

6.2.4 run

FPGA state	Triggered by	Next state on success	Next state on failure
reserved	Worker sends payload over UDP/TCP to FPGA IP	reserved (unchanged, session TTL renewed)	reserved (unchanged, error returned to user)

i Note

The FPGA state does not change on a run job regardless of outcome. The board stays reserved; only the job result and session TTL are updated.

6.2.5 reset

FPGA state	Triggered by	Next state on success	Next state on failure
reserved	Worker flashes base SoC bitstream over JTAG	idle (session destroyed, board available)	error (JTAG failed during reset)
error	Admin-triggered recovery reset	idle	error (board still unresponsive)

6.2.6 Worker Resources

A key design decision deals with needed resources per worker. 10 workers implies 10 concurrent processes. 10 `build_and_program` jobs will likely be computationally expensive. Based on host resources, a semaphore will be used to limit concurrent builds. Only a decided upon fraction of the 10 FPGAs will be able to run intensive jobs concurrently based on resources available.

6.3 Build Server

The build server denotes the functions executed in a `build_and_program` job. It follows a five stage pipeline.



Figure 6: Build pipeline: Amaranth HDL → bitstream

Stage	Input	Tool	Output	Key failure modes
1 — Amaranth → Verilog	<code>design.py</code> (user Amaranth source)	Amaranth HDL	<code>user_design.v</code>	Syntax error in user code, undefined signal, invalid module structure, module signature mismatch

Stage	Input	Tool	Output	Key failure modes
2 — LiteX SoC merge	<code>user_design.v</code> + base SoC definition	LiteX builder + Yosys <code>read_verilog</code>	<code>top.v</code> (combined Verilog)	UserDesign module name mismatch, Wishbone port mismatch, clock domain error
3 — Yosys synthesis	<code>top.v</code>	Yosys	<code>netlist.json</code> (gate-level netlist)	Unresolved signals, LUT budget exceeded, unsupported Verilog constructs
4 — nextpnr place and route	<code>netlist.json</code> + constraints file	nextpnr-ecp5	<code>design.config</code>	Design unroutable (too dense), timing closure failure, invalid pin assignment
5 — ecppack bitstream	<code>design.config</code>	ecppack	<code>bitstream.bit</code>	Malformed config file (rare, usually indicates nextpnr bug)

Open Design Decision

The HDL-to-Verilog conversion strategy is not finalized. The initial design assumes Amaranth HDL, but users should ultimately be able to choose their HDL.

6.3.1 Sandboxing

There remains multiple ways to merge Amaranth with a LiteX SoC. For the simplicity of sandboxing, we will start with converting Amaranth to Verilog and referencing a predefined user template in the SoC, allowing Yosys to resolve them.

Sandboxing a job prevents malicious behavior accessing non-permitted resources. We will start with a restricted subprocess with defined resource limits for initial prototyping. We plan to use Docker in later design iterations.

6.4 JTAG Programmer

1. Identifies USB serial for target FPGA.
2. Calls `openFPGALoader` and waits for completion.
3. Pings FPGA's IP address to confirm running.

6.5 Runtime Bridge

The runtime bridge handles `POST /run` request from the API server. It opens a TCP/UDP socket and sends the payload, waiting for result unless a timeout occurs.

Open Design Decisions

- UDP vs. TCP for brief control messages and persistent runtime data transfers is undecided.
- Error handling strategy across the host layer is undecided.

7 Networking

7.1 Local LAN Topology

10 FPGAs exist on a local LAN. The ethernet switch described in the physical layer section contains the FPGAs on a subnet with a 24-bit mask `/24`. The following table describes their addresses.

Device	Interface	IP address	MAC address	Notes
Host	host_eth_x	DHCP from router	Assigned by OS	internet-facing
Host	host_eth_x	192.168.1.10/24	Assigned by OS	FPGA-facing, Static, configured via Netplan
FPGA 1	LAN8720A	192.168.1.101	02:00:00:00:00:01	Hardcoded in SoC bitstream
FPGA 2	LAN8720A	192.168.1.102	02:00:00:00:00:02	Hardcoded in SoC bitstream
FPGA 3	LAN8720A	192.168.1.103	02:00:00:00:00:03	Hardcoded in SoC bitstream
FPGA 4	LAN8720A	192.168.1.104	02:00:00:00:00:04	Hardcoded in SoC bitstream
FPGA 5	LAN8720A	192.168.1.105	02:00:00:00:00:05	Hardcoded in SoC bitstream
FPGA 6	LAN8720A	192.168.1.106	02:00:00:00:00:06	Hardcoded in SoC bitstream
FPGA 7	LAN8720A	192.168.1.107	02:00:00:00:00:07	Hardcoded in SoC bitstream
FPGA 8	LAN8720A	192.168.1.108	02:00:00:00:00:08	Hardcoded in SoC bitstream
FPGA 9	LAN8720A	192.168.1.109	02:00:00:00:00:09	Hardcoded in SoC bitstream
FPGA 10	LAN8720A	192.168.1.110	02:00:00:00:00:0a	Hardcoded in SoC bitstream

i Note

The host ethernet interface name (`host_eth_x`) is not yet defined and depends on the specific host system.

7.1.1 ARP

An [ARP table](#) is used to map FPGA MAC addresses to IP addresses. We may consider creating a static ARP table as the MAC values are always known. This disambiguates different networking errors.

7.1.2 FPGA Interconnections

The network layer does not isolate FPGAs from communicating between one another in the initial design. This is handled by software. This may be added in future iterations.

7.1.3 Firewall

1. Host to Internet: The host should block inbound connections. DigitalOcean Droplet will be interfaced with the host via Tailscale.
2. Host to FPGA: The host should block against ports that will be decided on unnecessary for conducting FPGA jobs as to block against malicious FPGA usage targeting the host.

i Note

More specific firewall rules will be defined at implementation time.

7.2 Remote Access

Remote access to the host machine will be done via a [DigitalOcean Droplet](#) and a [TailScale](#) mesh network.

7.2.1 DigitalOcean Droplet

The Droplet is a minimal hosted virtual machine (VM). It acts as the traffic forwarder between the internet and our host machine. The most affordable options are illustrated below.

Plan	CPU type	vCPUs	Memory
Basic	Shared	1–8	1–32 GB RAM
General Purpose	Dedicated	2–48	8–240 GB RAM (4 GB RAM / vCPU)

[nginx](#) acts as a reverse proxy that forwards the internet requests to the host. This will exist on the VM. We will need to decide on rate limitations.

For security, HTTPS will be used via [Certbot](#) which provides Transport Layer Security (TLS) certificate management. A domain name will be needed for a TLS certificate.

The Droplet will need to run a TailScale client to connect to TailScale.

7.2.1.1 Firewall

The droplet should only expose HTTPS and HTTP from anywhere on the internet. Only SSH should be exposed from the host for maintenance.

7.2.2 TailScale

A TailScale tailnet will securely connect the Droplet to the host machine.

7.3 Runtime Datapath

7.3.1 To FPGA

Step	From	To	Protocol	Key decision
1	User client	Droplet nginx	HTTPS port 443	Payload encoding.
2	Droplet nginx	Host API server	HTTP over Tailscale tunnel	Timeouts. User IP forwarding for logging
3	Host API server	Redis	Internal	Validate session ownership, FPGA state, and API key before enqueueing. All checks are Redis reads
4	Host worker	FPGA static IP	UDP or TCP port XXXX	New socket per request. Protocol Decision Per Job. Packet Wrapping Format
5	Ethernet switch	FPGA 3 LAN8720A PHY	Layer 2 ethernet frame	Switch forwards based on MAC address. No routing involved
6	LiteEth MAC	VexRiscv firmware	DMA to SRAM then poll	MAC validates frame and CRC. DMA writes payload to SRAM. Firmware reads and parses header
7	VexRiscv firmware	User design slave	Wishbone bus at 0XXXXXXXXX	Firmware issues write transaction. User design processes and asserts ACK

7.3.2 From FPGA

Step	From	To	Protocol	Key decision
8	User design slave	VexRiscv firmware	Wishbone ACK + DAT_R	Result read by firmware. Reply packet constructed with status byte + length + data
9	FPGA LiteEth MAC	Host worker socket	UDP or TCP port XXXX	Same socket receives reply. Latency measured from send to receive
10	Host worker	Redis + API server	Internal	Result written to Redis. Job marked complete. Session TTL renewed

Step	From	To	Protocol	Key decision
11	Host API server	Droplet nginx	HTTP over Tailscale tunnel	JSON response with result, latency, job ID, session TTL renewed confirmation
12	Droplet nginx	User client	HTTPS port 443	TLS re-applied. User receives decoded result and latency

7.4 Protocols and Ports

Service	Protocol	Port	Interface	Accessible from	Notes
HTTPS — user API	TCP	443	Droplet public IP	Anywhere	nginx proxies to host
HTTP redirect	TCP	80	Droplet public IP	Anywhere	nginx redirects to 443
SSH — Droplet admin	TCP	22	Droplet public IP	Your IP only	Key-Based Auth
SSH — host admin	TCP	22	tailscale0	Tailscale mesh only	Never exposed on internet-facing interface
Host API server	TCP	8000	tailscale0	Droplet only	FastAPI bound to Tailscale IP
Redis	TCP	6379	localhost only	Host processes only	Never exposed on any network interface
FPGA runtime data	UDP or TCP	XXXX	(FPGA LAN)	Host worker only	Port is fixed in firmware and host config
FPGA readiness check	ICMP	—	(FPGA LAN)	Host only	Ping after programming to confirm boot
Tailscale tunnel	UDP	41641	All interfaces	Tailscale coordination servers	Managed automatically by Tailscale client

8 User Interface Layer

8.1 HDL Interface Contracts

Contract element	Requirement
Module name	A design name must be decided upon that matches a referenced module in the SoC.
Top-level file	Must be a single <code>.py</code> file containing the user module. Assuming solely Amaranth for now.
Bus interface name	Must be same name used in LiteX SoC.
Address width	Address width must be decided based on bus width.
Data width	32 bits
Required signals	Standard Wishbone B4 slave signals
Base address	Hardcoded in SoC; user design is always mapped here
Address range	Decide on range exclusively owned by user design
Forbidden addresses	Any address used for SoC infrastructure

8.2 User Submission Contracts

Contract element	Requirement
File format	Single <code>.py</code> file containing user module for initial design.
Maximum file size	Maximum submission size must be decided on. Exceeding this limit emits API rejection.
Available HDL framework Libraries	amaranth — HDL library for initial design. Standard Library and External Packages open to design decision.
Network access	None — the build sandbox has no internet access
Filesystem access	None — the build sandbox cannot read or write outside the build directory
Multiple files	Not supported in v1 — all design code must be contained in the single submitted file

8.3 User Return Contracts

Contract element	Requirement
Successful build response	Job ID, session ID, FPGA ID, session expiry timestamp
Build failure response	Stage that failed, toolchain error message, link to full build log
Build log retention	Design decision on time from job completion — retrievable via <code>GET /fpga/{id}/jobs/{job_id}/logs</code>
Runtime success response	Result, latency in milliseconds, renewed session TTL
Runtime failure — FPGA timeout	Returned as 504 with error: fpga_timeout — infrastructure problem, FPGA did not respond
Runtime failure — solver timeout	Returned as 504 with error: solver_timeout — user timeout expired, problem may be solvable with more time
Runtime failure — session expired	Returned as 409 with error: session_expired — user must resubmit design
Runtime failure — wrong state	Returned as 409 with error: fpga_not_reserved — FPGA not in reserved state
Result storage	Results are returned once in the HTTP response and not stored — client must handle network errors and resubmit if result is lost
Session expiry warning	Not provided — client is responsible for checking remaining TTL via <code>GET /fpga/{id}/session</code> before it expires
Error format	All errors return JSON with error (machine-readable code), message (human-readable description), and job_id where applicable

8.4 Runtime Contracts

Polling intervals, retry behaviors, and session renewals are left to when we handle overlaying competition software.

9 Infrastructure Does Not Guarantee

Contract element	Guarantee
Design persistence	No — when a session expires or is reset the user design is gone. Users must resubmit and rebuild to reuse a design
Bitstream storage	No — the infrastructure does not retain bitstreams after a session ends
Inter-FPGA communication	Not supported — FPGAs are independent nodes and cannot communicate with each other
FPGA availability	Not guaranteed — if all 10 FPGAs are reserved or in error state the user must wait with no SLA on when one becomes available
Result storage	Not guaranteed — results are returned once in the HTTP response and not retained by the infrastructure
Build time	Not guaranteed — nextpnr place and route for a large design can take a long time. Workflows must tolerate variable build duration
Session continuity across host reboot	Not guaranteed — if the host machine reboots, active sessions are invalidated and FPGAs may return to idle or error state
Concurrent FPGA access	Not supported — one session per FPGA at a time. A user cannot hold multiple FPGAs simultaneously in v1. We will implement this in the next iteration.
Partial result on timeout	Not provided — if a run job times out no partial result is returned. The user must resubmit the problem with a longer timeout

References

- [1] Amazon Web Services. *Amazon EC2 F2 Instances HDK README*. <https://awsdocs-fpga-f2.readthedocs-hosted.com/latest/hdk/README.html>. Accessed 2026. 2024.
- [2] Analog Devices. *LT3508: Dual Monolithic 1.4A Step-Down Switching Regulator Datasheet*. Rev. E. 2024.
- [3] Lattice Semiconductor. *ECP5 Evaluation Board User Guide*. FPGA-EB-02017-1.3. Apr. 2023.
- [4] Jiaqi Liu et al. “AutoResearchClaw: Self-Reinforcing Autonomous Research with Human-AI Collaboration”. In: *arXiv preprint arXiv:2605.20025* (2026).
- [5] Michael Lo, Mau-Chung Frank Chang, and Jason Cong. “SAT-Accel: A Modern SAT Solver on a FPGA”. In: *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. FPGA ’25. Monterey, CA, USA: ACM, 2025, pp. 234–246. DOI: [10.1145/3706628.3708869](https://doi.org/10.1145/3706628.3708869).
- [6] Microchip Technology Inc. *LAN8720A/LAN8720AI: Small Footprint RMII 10/100 Ethernet Transceiver with HP Auto-MDIX Support*. DS00002165B. 2016.
- [7] OpenCores. *Wishbone B4: WISHBONE System-on-Chip (SoC) Interconnection Architecture for Portable IP Cores*. 2010.
- [8] Buse Ustaoglu et al. “SAT-Hard: A Learning-based Hardware SAT-Solver”. In: *2019 22nd Euromicro Conference on Digital System Design (DSD)*. IEEE, 2019.
- [9] Qingyun Zou et al. “Is Agentic AI Ready for Real-World Hardware Engineering? A Deep Dive with Phoenix-bench”. In: *arXiv preprint arXiv:2605.15226* (2026).